10

Part 10 · Interview Toolkit

Checklist, errors, config cheat sheet, answer framework, Q&A and glossary.

- Optimization checklist
- Common errors & fixes
- Config cheat sheet
- Answering "slow job"
- Rapid-fire Q&A
- Glossary

10.1 The Spark Optimization Checklist

★★★

IN ONE LINE

Seven ordered priorities: read less, shuffle less, balance, fit in memory, reuse, lay out data, tune the platform.

If you remember one thing from this handbook, make it this **ordered list**. Interview answers and real tuning both go from top to bottom: first stop doing unnecessary work, then make the remaining work efficient, and only then adjust infrastructure.

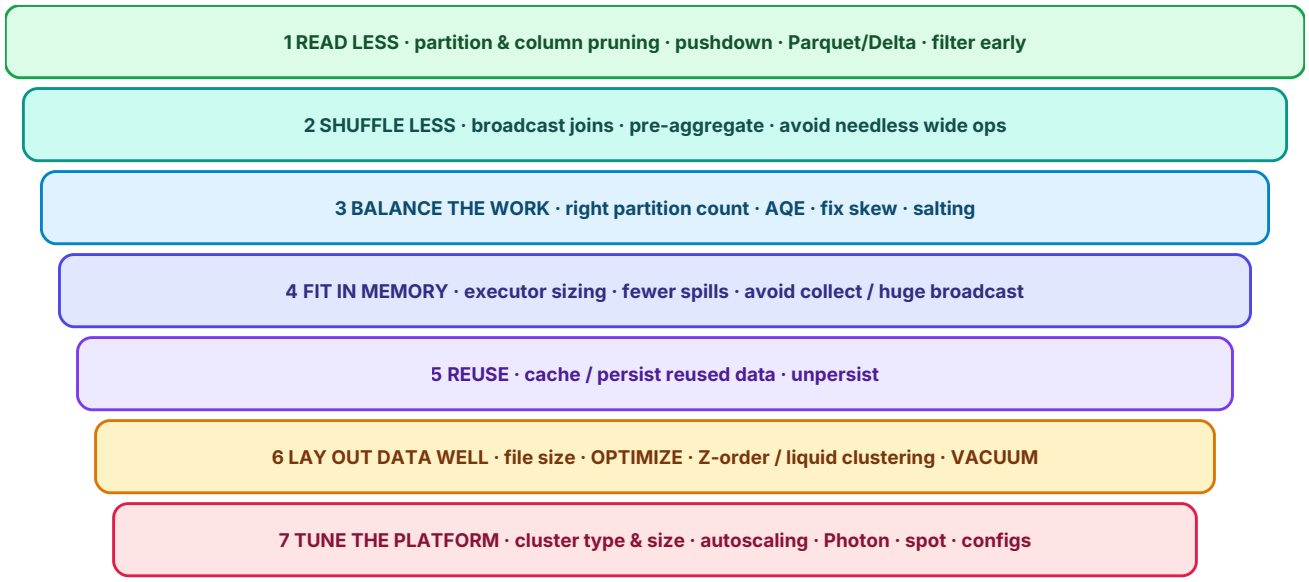


Figure 10.1 — Spark optimisation priorities

If the problem is...	First reach for...	See
Slow scans, big input	Partition / column pruning, Parquet/Delta, compaction	2.6 – 2.9, 5.x
Join is slow	Broadcast, filter first, AQE, check explain()	3.1 – 3.3, 6.3
One task never finishes	AQE skew, salting, filter hot keys	3.5 – 3.7
Spill or OOM	More partitions, memory per core, no collect	2.5, 4.4 – 4.6
Same work repeated	cache() the reused DataFrame (and unpersist)	4.1 – 4.3
Many tiny files / tasks	Compaction, OPTIMIZE, fewer shuffle partitions	5.4 – 5.6, 6.2
Costs too high	Do less work, job compute, spot, autoscaling	7.x, 9.5

10.2 Common Errors and What to Do

★★★

IN ONE LINE

Typical Spark failures, their causes and first fixes.

These are the error messages and symptoms you will meet again and again, with the usual cause and first fix. Interviewers often present one and ask “what do you do?”.

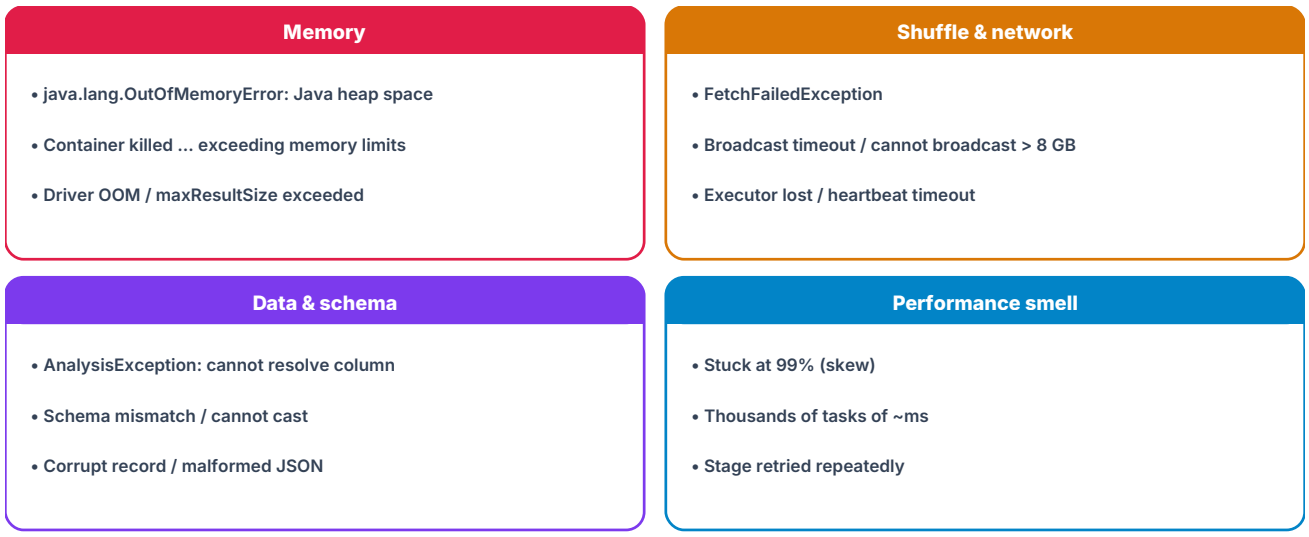


Figure 10.2 — Common errors, grouped

Error / symptom	Usual cause	First fix
Java heap space (executor)	Task data larger than its memory: skew, big partitions, huge groups	More partitions, fix skew, more memory per core
Container killed by YARN / K8s	Off-heap use exceeds limit (PySpark, pandas UDFs, buffers)	Raise `memoryOverhead`; reduce Python-side data
Driver OOM / maxResultSize	`collect()`, `toPandas()`, big broadcast	Aggregate first; write to storage; raise only if necessary
FetchFailedException	Shuffle files lost (executor died, spot loss, overload)	Fix memory pressure; retry; external shuffle service / stable nodes
Broadcast timeout / too large	Broadcasting a big or slow side	Remove hint / lower threshold; raise timeout only if sure
Stuck at 99%	Skewed task, or tiny straggler waiting on I/O	Check task metrics; AQE / salting
Too many tasks / slow listing	Small files, over-partitioned table	Compact, OPTIMIZE, better partition column
AnalysisException cannot resolve	Wrong / ambiguous column after join, case issues	Alias columns, qualify names, check schema

INTERVIEW ANGLE

Pattern for answering: **symptom → likely cause → how I would confirm in the Spark UI → fix**. For OOM always ask *where* it happened: driver or executor.

10.3 Configuration & Code Cheat Sheet

★★★

IN ONE LINE

Key Spark settings with defaults, plus a one-screen set of optimisation snippets.

Defaults below are from the Apache Spark 4.0 configuration reference; managed platforms (Databricks) may override some. Know the **defaults** and **why you would change them**.

Setting	Default	What it does
spark.sql.shuffle.partitions	200	Partitions after a shuffle; tune or rely on AQE
spark.sql.adaptive.enabled	true	Runtime re-optimisation (AQE)
spark.sql.adaptive.coalescePartitions.enabled	true	Merge small shuffle partitions
spark.sql.adaptive.advisoryPartitionSizeInBytes	64 MB	Target post-shuffle partition size
spark.sql.adaptive.skewJoin.enabled	true	Split skewed join partitions
spark.sql.adaptive.skewJoin.skewedPartitionFactor	5.0	Skew if > factor × median
spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes	256 MB	and larger than this size
spark.sql.autoBroadcastJoinThreshold	10 MB	Max size to auto-broadcast; -1 disables
spark.sql.files.maxPartitionBytes	128 MB	Max bytes per read partition
spark.sql.parquet.compression.codec	snappy	Parquet compression codec
spark.sql.optimizer.dynamicPartitionPruning.enabled	true	Prune partitions via join filters
spark.memory.fraction	0.6	Share of heap for execution + storage
spark.memory.storageFraction	0.5	Protected part of unified memory for storage
spark.executor.memoryOverhead	max(10%, 384 MB)	Non-heap memory per executor
spark.dynamicAllocation.enabled	varies	Add / remove executors with load
spark.sql.ansi.enabled	true (Spark 4)	Strict SQL semantics: errors instead of nulls

PYSPARK ONE-SCREEN CHEAT SHEET

```
# the 10 lines that fix most jobs
spark.conf.set('spark.sql.adaptive.enabled', True)                # AQE on
spark.conf.set('spark.sql.shuffle.partitions', 800)               # generous; AQE coalesces
large.join(F.broadcast(small), 'key')                             # avoid big shuffle
df.select('a', 'b').filter('dt = "2026-10-19"')                   # prune + push down
df.repartition(200, 'key')                                        # balance / co-locate
df.coalesce(8).write.parquet(path)                                # fewer output files
df.cache(); df.count(); ...; df.unpersist()                       # reuse then release
spark.read.schema(schema).csv(path)                               # no inference pass
df.explain('formatted')                                           # verify the plan
spark.sql('OPTIMIZE t ZORDER BY (col)')                           # compact + cluster (Delta)
```

INTERVIEW ANGLE

You do not need to memorise every key. Memorise the **defaults that interviewers quote**: shuffle partitions 200, broadcast threshold 10 MB, read partition 128 MB, AQE on, skew factor 5 / 256 MB, storage 50% of 60% unified memory.

10.4 How to Answer a "Slow Spark Job" Question

★★★

IN ONE LINE

A six-step structure and model answers for the most common scenarios.

Most interviews end with an open question about a slow or failing job. A fixed structure keeps you calm and shows **engineering method**, which interviewers value above memorised trivia.

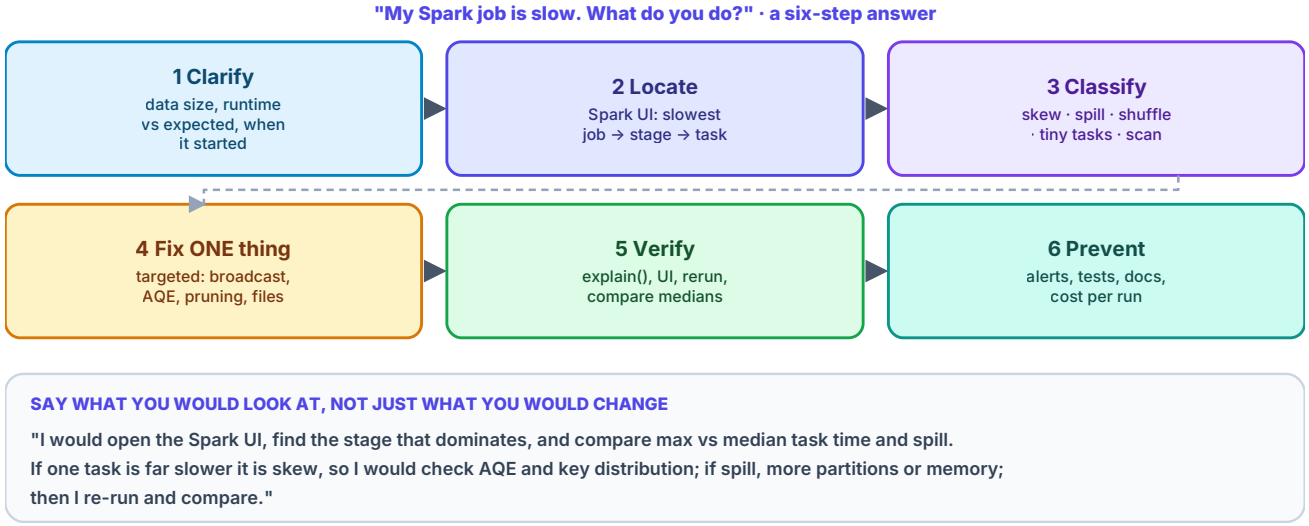


Figure 10.3 — A repeatable six-step answer

Scenario	A strong answer covers
Job stuck at 99%	Check task metrics: one huge partition. Confirm key skew; AQE skew join, broadcast if possible, else salting; filter NULL / default keys.
Job fails with OOM	Driver or executor? Driver: remove collect / giant broadcast. Executor: more partitions, fix skew, more memory per core, raise memoryOverhead for PySpark.
Join of two big tables is slow	Filter and select first; is one side small after filtering? broadcast. Check duplicates in keys; AQE on; co-locate with clustering for repeated joins.
Table queries got slower over weeks	Small files from appends / streaming: OPTIMIZE (+ Z-order / clustering), enable auto-compaction, VACUUM, check partition strategy.
Streaming job falling behind	Process rate < input rate? Cap batch size, tune partitions, check state size / watermark, skew, sink small files, scale cluster.
Costs doubled	What changed: data volume, cluster type, a code change? Job vs all-purpose compute, autoscaling, idle clusters, wasteful scans.

INTERVIEW ANGLE

Finish with **trade-offs**: "broadcast saves a shuffle but uses memory; more partitions reduce spill but increase overhead; Photon speeds up but costs more per hour." Naming the trade-off proves understanding.

10.5 Rapid-Fire Interview Q&A

★★★

IN ONE LINE

Thirty-five questions with answers short enough to say out loud.

Q What is lazy evaluation and why does Spark use it?

Transformations only build a plan; nothing runs until an action. It lets Catalyst optimise the whole query, skip unneeded work and pipeline operations.

Q Transformation vs action? Give examples.

Transformations (select, filter, groupBy, join) lazily define a new DataFrame; actions (count, show, collect, write) trigger execution and return or write results.

Q Explain job, stage and task.

An action creates a job; a job is cut into stages at shuffle boundaries; each stage runs one task per partition on an executor core.

Q Narrow vs wide transformation?

Narrow: each output partition depends on one input partition (filter, select). Wide: needs data from many partitions, so a shuffle (groupBy, join, distinct).

Q Why is shuffle expensive?

It writes intermediate data to disk, serialises it, moves it across the network, then sorts/merges it, and creates a stage boundary.

Q repartition vs coalesce?

repartition does a full shuffle and can increase or decrease partitions evenly; coalesce only merges existing partitions without a shuffle (decrease only, may be uneven).

Q How do you pick spark.sql.shuffle.partitions?

Roughly shuffle data size ÷ 128 MB and at least 2-3× the cores; with AQE set it high and let coalescing shrink it. Default is 200.

Q What is Catalyst?

Spark SQL's optimiser: rule-based and cost-based rewrites such as predicate pushdown, column pruning and join selection. Tungsten handles efficient execution.

Q Predicate pushdown vs partition pruning vs column pruning?

Pruning skips whole partition folders (partition column filter) or unneeded columns (columnar format); pushdown sends filters to the source to skip row groups or rows.

Q How does a broadcast join work and when do you use it?

The small table is sent to every executor and joined locally, so the big table is not shuffled. Use when one side is small (default under 10 MB, configurable; hard limit ~8 GB).

Q Sort-merge vs broadcast vs shuffled hash join?

Broadcast: no shuffle, small side in memory. Sort-merge: shuffle + sort both sides, default for big-big. Shuffled hash: shuffle both, build a hash table on the smaller side, no sort.

Q What is data skew and how do you handle it?

Uneven key distribution making a few tasks huge. Use AQE skew join, broadcast, filter hot / NULL keys, salting, or isolate the hot key.

Q How does salting work?

Add a random salt to the hot key on the big side and replicate the small side across all salt values; join on (key, salt) to spread one key over many tasks.

Q What does AQE do?

Re-optimises the plan at runtime using real shuffle statistics: coalesces small partitions, switches join strategies (e.g. to broadcast) and splits skewed partitions.

Q cache vs persist?

cache() is persist() with the default level (MEMORY_AND_DISK for DataFrames). persist lets you choose storage level. Both are lazy; unpersist releases them.

Q When should you NOT cache?

When the DataFrame is used once, is cheap to recompute, or is too big for memory; caching wastes memory and can slow the job.

Q Why does df.cache(); df.show() not fully cache?

show() computes only the partitions it needs, so only those are cached. Use count() to materialise every partition.

Q Explain Spark memory management.

Heap = 300 MB reserved + user memory + unified memory (60%) shared by execution and storage (50% protected for storage), plus off-heap overhead (max 10% / 384 MB).

Q What is spill and how do you reduce it?

Writing intermediate data to disk when a task exceeds its memory. Reduce with more partitions, fixing skew, or more memory per core.

Q Difference between OOM on driver vs executor?

Driver: collect, toPandas, huge broadcast or too many partitions. Executor: oversized tasks from skew/big partitions or too little memory per core.

Q Why prefer Parquet / Delta over CSV / JSON?

Columnar, compressed, typed, splittable, supports pruning and pushdown; Delta adds ACID, MERGE, time travel and data skipping.

Q Why define schemas explicitly?

Avoid the extra inference scan, wrong type guesses and unstable schemas; control bad-record handling.

Q What is the small files problem?

Many tiny files add per-file overhead (listing, open, task scheduling). Fix by compaction, OPTIMIZE, fewer shuffle partitions at write and avoiding high-cardinality partitions.

Q What does Delta OPTIMIZE do?

Compacts small files into larger ones (around 1 GB target) transactionally; with ZORDER it also co-locates data for better skipping.

Q What is Z-ordering?

A multi-column sort layout in Delta that clusters related values into the same files so min/max stats skip more files for filters on those columns.

Q VACUUM: what and why careful?

Deletes files no longer referenced and older than retention (default 7 days). Too short retention breaks time travel and in-flight readers.

Q Liquid clustering vs partitioning + Z-order?

Liquid clustering (CLUSTER BY) is a flexible incremental layout recommended for new Delta tables; keys can change without rewriting everything; not combined with partitioning/Z-order.

Q Why are Python UDFs slow?

Rows are serialised between the JVM and Python and the UDF is opaque to Catalyst (no pushdown / codegen). Prefer built-ins, then pandas UDFs.

Q What does Photon do?

A vectorised C++ engine on Databricks that accelerates supported SQL/DataFrame operators with no code change, at a higher DBU rate.

Q Job cluster vs all-purpose cluster?

Job compute is created per run, cheaper and isolated, for production; all-purpose is shared interactive compute for development.

Q What do cluster pools give you?

Pre-warmed idle instances so clusters start and scale much faster; idle instances still cost VM money.

Q How does Structured Streaming achieve exactly-once?

Replayable source + checkpoint (offsets and commits) + idempotent or transactional sink such as Delta.

Q What is a watermark?

A threshold on event time telling Spark how late data can be; windows older than the watermark are finalised and their state dropped.

Q What is Auto Loader?

Databricks incremental file ingestion (cloudFiles) that tracks processed files, supports schema inference/evolution and rate limits, with exactly-once semantics.

Q How do you find why a job is slow?

Spark UI: slowest stage, compare max vs median task time, check spill, shuffle sizes, GC, then SQL tab for the operator; fix one thing and re-measure.

Q How do you benchmark a change?

Same data and fixed cluster, repeat runs, compare medians, control caches, verify output equality, and consider cost per run.

10.6 Glossary: Every Term in One Line

★★☆

IN ONE LINE

A quick-revision index of all concepts in this handbook.

Every term in the roadmap plus the extras, in one place. The last column points to the section with the full explanation and diagram.

Term	One-line definition	§
Action	Operation that triggers execution and returns / writes a result.	1.4
Adaptive Query Execution (AQE)	Runtime re-optimisation using shuffle statistics.	6.1
Advisory partition size	AQE target size for coalesced partitions (64 MB).	6.2
Auto Loader	Incremental cloud-file ingestion on Databricks.	8.4
Autoscaling	Adding / removing workers with load.	7.2
Benchmarking	Controlled before/after measurement.	9.2
Broadcast join	Small table copied to every executor; no shuffle of big side.	3.2
Bucketing	Pre-hashing data on a key into a fixed number of files.	2.9
Cache / persist	Keep computed data in memory / disk for reuse.	4.1
Catalyst	Spark SQL query optimiser.	1.8
Checkpoint (streaming)	Durable record of offsets, commits and state.	8.3
Cluster pool	Pre-warmed idle instances to cut start-up time.	7.5
Coalesce	Reduce partitions without a full shuffle.	2.4
Column pruning	Reading only needed columns.	2.7
Compaction	Merging small files into larger ones.	5.5

Term	One-line definition	§
Compression codec	Snappy, zstd, gzip, LZ4 file compression.	5.3
Cost-based optimizer (CBO)	Uses statistics to choose join order / strategy.	6.6
DAG	Directed acyclic graph of operations.	1.5
Data skew	Uneven key distribution creating straggler tasks.	3.5
Data skipping	Skipping files using min/max statistics.	5.7
Databricks Runtime (DBR)	Managed Spark + Delta + libraries image.	7.6
DataFrame	Distributed table with a schema.	1.2
Delta Lake	Parquet + transaction log: ACID tables.	5.1
Disk cache	Local-SSD cache of remote files on Databricks.	7.7
Driver	Process that plans and coordinates a Spark app.	1.1
Dynamic partition pruning	Runtime partition filters from join conditions.	2.6
Event time / watermark	When an event occurred / lateness threshold.	8.2
Executor	JVM process that runs tasks and holds cache.	1.1
explain()	Prints logical / physical plans.	6.5
GC (garbage collection)	JVM memory reclamation pauses.	4.4
Hash partitioning	Rows assigned by hash(key) mod N.	2.1
Job	Work triggered by one action.	1.6
Lazy evaluation	Plans are built, executed only on actions.	1.3
Liquid clustering	Flexible incremental Delta data layout (CLUSTER BY).	7.7
Logical / physical plan	What to compute / how to compute it.	1.7
Micro-batch	Small incremental batch processed by a stream.	8.1
Narrow / wide transformation	No shuffle / shuffle required.	2.2
OPTIMIZE	Delta file compaction command.	5.6
Pandas UDF	Vectorised Python UDF using Arrow.	3.8
Partition	Chunk of data processed by one task.	2.1
Partition pruning	Skipping folders via partition column filters.	2.6
Photon	Databricks vectorised C++ engine.	7.3
Predicate pushdown	Sending filters to the data source.	2.8
Predictive optimization	Automatic OPTIMIZE / VACUUM on managed tables.	7.7
RDD	Low-level resilient distributed dataset.	1.2
repartition	Full shuffle to a given partition count / key.	2.4
Salting	Random key suffix to spread a hot key.	3.7
Shuffle	All-to-all data exchange between stages.	2.3
Shuffle partitions	Partition count after a shuffle (default 200).	2.5
Small files problem	Per-file overhead from many tiny files.	5.4

Term	One-line definition	§
Sort-merge join	Default join for large tables: shuffle + sort + merge.	3.1
Spark UI	Web UI for jobs, stages, tasks, SQL.	1.9
Spill	Writing working data to disk when memory is short.	4.5
Stage	Tasks between shuffle boundaries.	1.6
State store	Streaming state for windows / joins.	8.2
Storage level	Where / how persisted data is kept.	4.2
Structured Streaming	Incremental processing on the DataFrame API.	8.1
Task	Unit of work on one partition.	1.6
Transformation	Lazy operation returning a new DataFrame.	1.4
Tungsten	Memory / CPU-efficient execution engine.	1.8
UDF	User-defined function (slow if Python row-by-row).	3.8
Unified memory	Shared execution + storage heap region.	4.4
VACUUM	Deletes obsolete Delta files after retention.	5.8
Z-ordering	Multi-column data clustering for skipping.	5.7